CS 552 · GROUP PROJECT | IMPLEMENTATION & EVALUATION

An Efficient Framework for Phishing URL Detection Using Adaptive Drift-Aware Feature Monitoring and Online Learning

Implementation and evaluation — CS 552 Group Project

Pratham Garg | **Sumit Sharma** | **Vivaan Garg**

ASSIGNED PAPER — FULL REFERENCE

Sruthi Krishnan and Manohar Naik S. “An Efficient Framework for Phishing URL Detection Using Adaptive Drift-Aware Feature Monitoring and Online Learning.” *Security and Privacy*, vol. 8, no. 4, article e70057, 2025. John Wiley & Sons Ltd.

Department of Computer Science, Central University of Kerala, Periye, India.

DOI
10.1002/spy2.70057

PAPER LINK
https://onlinelibrary.wiley.com/doi/full/10.1002/spy2.70057

ROADMAP

Our 20-minute story: from “why?” to “does it work?”

1

The problem

Phishing links and why a model trained once goes stale (concept drift).

≈ 3 min

2

The paper’s idea

URL-only features + a model that watches for drift and updates itself.

≈ 2 min

3

How it works

Step by step: features → LASSO + MI → SGD-SVM → KS drift test → online loop.

≈ 6 min

4

What we built

Our Python implementation, data we used, experiments we ran.

≈ 2 min

5

Results

What reproduced, what did not, and an honest look at why.

≈ 6 min

THE ONE-LINE VERSION

A phishing detector that notices when attackers change their style, re-picks the most useful clues, and keeps learning in milliseconds instead of retraining from scratch.

THE PROBLEM

Phishing starts with a link that looks almost right

https://www.paypal.com/signin — real

Real login page — you type your password, PayPal gets it.

http://secure-paypal-login.tk/confirm/account?user=1 — phishing

Looks the same on screen — the attacker gets your password.

WHY THIS IS HARD

Attackers send millions of such links by email, SMS and social media. Blocklists are always one step behind a brand-new link — so we need a model that can judge a link **it has never seen**.

TWO WAYS TO JUDGE A LINK

	PAGE-CONTENT METHODS	URL-ONLY (THIS PAPER)
Needs	download + render the page	**just the text of the URL**
Speed	slow	**instant**
Privacy	reads page content	**no page content read**
Safety	touches the malicious site	**never visits it**

KEY QUESTION

Can the text of a URL alone be enough — and can the model stay accurate as attacks evolve?

THE CATCH · CONCEPT DRIFT

A model trained once goes stale — we measured it

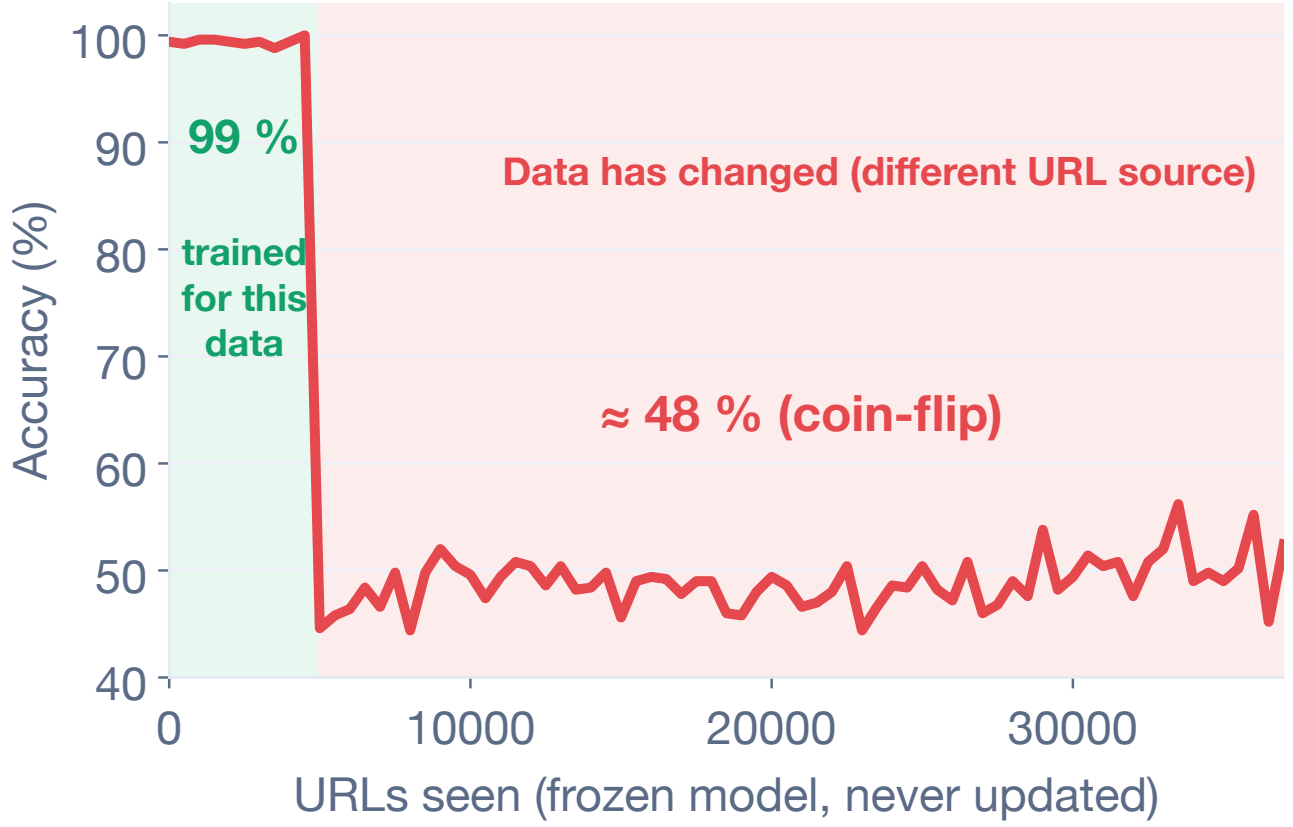
What is “concept drift”?

The statistics of the data **change over time**. Phishers keep changing how their URLs look — shorter hostnames, more random characters, fewer bait words like “free” or “secure”.

A model learned from *yesterday’s* URLs makes more and more mistakes on *today’s*. In our data **34 of 58** features measurably shifted between old and new phishing URLs.

THE OBVIOUS FIX — AND ITS COST

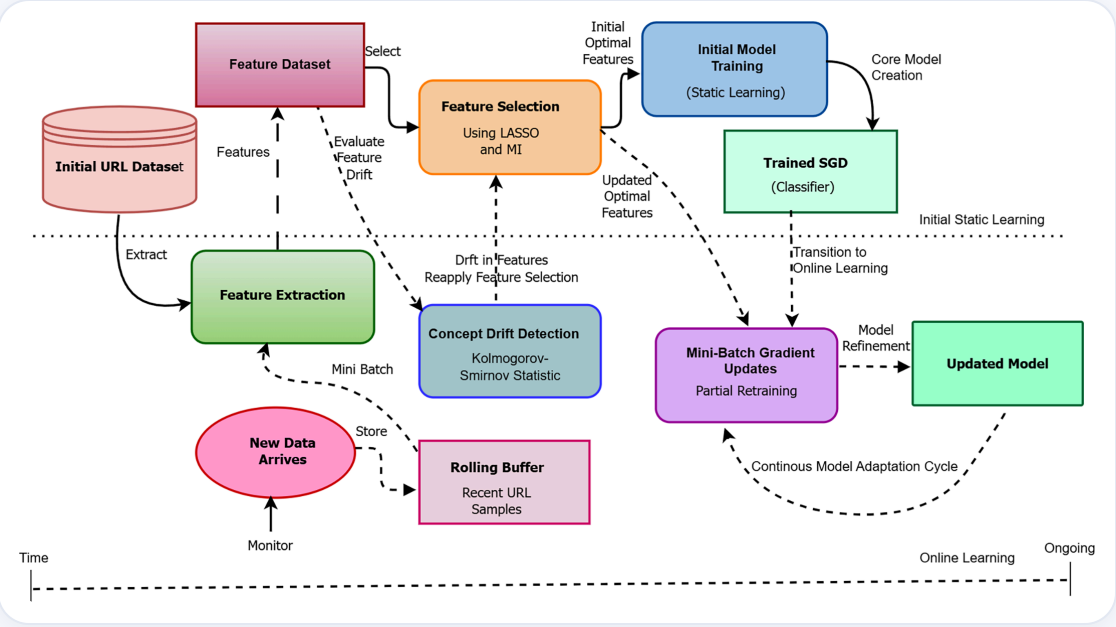
Retrain from scratch on all data each time: about **4 seconds** on 210k URLs in our runs, vs. **1.5 ms** for an incremental update.



Our experiment (S2): the same model that scores **99%** on the data it was trained for drops to **≈ 48%** — coin-flip level — once URLs come from a different source, because it is **frozen** and never updated.

THE PAPER'S IDEA

Three ingredients that keep a detector current



Paper's Fig. 1. Top: offline phase (done once). Bottom: online phase (never ends) — new data enters a rolling buffer, a drift detector watches it, and the feature set and the model are refreshed.

1

Pick the best clues (LASSO + MI)

Out of dozens of URL features, keep only the most informative quarter. Re-pick when things change.

2

Watch for change (Kolmogorov–Smirnov test)

Compare each feature's distribution in new URLs vs. recent history. Statistically different → drift alarm.

3

Learn on the fly (online SGD)

Update a linear model with small mini-batches of new URLs — no full retraining.

PAPER'S HEADLINE CLAIM

99.85 % accuracy · 0.067 % false-positive rate

OUR TASK

Our task: implement every box of the paper's Fig. 1

Assignment: implement the paper's methodology and reproduce its experiments as closely as possible. We wrote everything from scratch as the Python package `drift_phish` (~1,200 lines) + 5 experiment scripts + 16 unit tests + a browser demo.

PAPER BOX

URL → features

Table 3: lexical URL features, no HTML, no 3rd-party lookups

`features/extractor.py`

58 features

›

PAPER BOX

Feature selection

LASSO + mutual information, Algorithm 1

`selection/dynamic.py`

S = w + I, τ = P75

›

PAPER BOX

SGD classifier

Table 4: SVM / LR / Perceptron; offline fit + online *partial_fit*

`learning/online_sgd.py`

η_0 = 0.01, balanced

›

PAPER BOX

Drift detection

K–S test per feature, δ = 0.05, buffer 1500, B = 500

`drift/ks_monitor.py`

two-sample KS test

GLUES IT TOGETHER

`framework.py` the full online loop

predict → KS test → (drift? re-select features + partial retrain) → learn from the batch. Four system modes for fair comparison: **adaptive** (proposed), **online_static**, **frozen**, **periodic_batch**.

Evaluation code · `eval/metrics.py`, `splits.py`, `data.py`

One command · `make all` reproduces every number & figure

STEP 1 · FEATURES

Turning a URL into numbers a model can use

`http://secure-paypal-login.tk/confirm/account?user=1&id=88`

scheme · domain name · TLD · path · query

Real feature values from our extractor

FEATURE (WHAT IT COUNTS)	WIKIPEDIA.ORG	SECURE-PAYPAL-LOGIN.TK
`url_length` characters in total	39	**58**
`num_hyphens` "-" characters	0	**2**
`number_count` digits	0	**3**
`entropy` randomness of characters	3.78	**4.62**
`has_https` / `scheme_http`	1 / 0	**0 / 1**
`suspicious_tld` (.tk, .xyz …)	0	**1**
`suspicious_term_frequency` bait words	0	**4**
`brand_name_proximity` brand in URL	0	**1**

RESULT

Each URL becomes a **vector of 58 numbers**. Nothing is downloaded — it's pure string processing (≈ 0.16 ms per URL).

WHY IT WORKS

Phishing URLs tend to be longer, hyphen- and digit-heavy, more random, use `http`, odd TLDs and bait words.

STEP 1 · FEATURES

58 URL clues in six families (paper Table 3)

Length & size 8

How big is each part?

`url_length` `domain_length` `path_length` `query_length` `token_count` `longest_host_label` `tld_length` …

Structure & symbols 18

Dots, slashes, hyphens, “=”, “&”, “@”, “%” …

`num_dots` `num_slashes` `num_hyphens` `num_subdomains` `num_params` `special_char_count` `has_port` …

Digits & randomness 6

Random-looking strings are suspicious.

`number_count` `digit_ratio` `entropy` `host_entropy` `num_digits_path` `num_digits_query`

Bait words & brands 11

22 social-engineering terms from the paper.

`suspicious_term_freq` `brand_name_proximity` `has_login` `has_verify` `has_secure` `has_account` `has_free` …

Obfuscation & risky hosts 9

Tricks to hide the real destination.

`is_ip` `url_shortening` `suspicious_tld` `homoglyph_detection` `url_encoded` `javascript_data_uri` `suspicious_file_ext` …

Scheme & redirects 6

HTTP vs HTTPS, redirect hints, domain mismatch.

`has_https` `scheme_http` `multiple_redirects` `redirect_chain_complexity` `domain_mismatch` …

Honest note: the paper’s redirect-chain and domain-mismatch features need live HTTP lookups. We use **lexical proxies** (e.g. “does the URL contain `redirect=` or a second `http`?”) so everything stays URL-string-only.

STEP 2 · FEATURE SELECTION

Not every clue helps — keep the top quarter

58 features
every URL → 58 numbers

LASSO → weight w
"Drop clues that don't help the linear model." Useless weights shrink to exactly 0 (λ = 0.0001).

Mutual information → I
"How much does knowing this clue reduce uncertainty about phishing vs. legit?" Also catches non-linear clues.

S = w + I
combined score per feature

Keep S ≥ τ
τ = 75th percentile → F* = 15 features

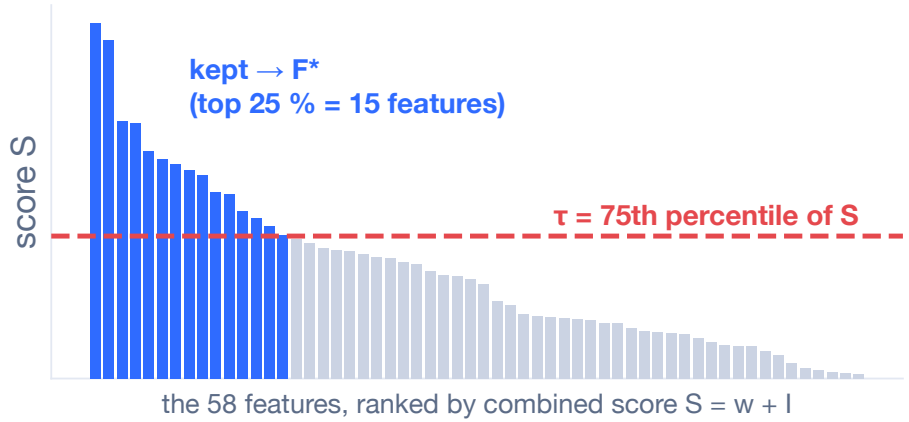


Illustration: rank all 58 features by score; the top quarter (blue) survives as the active feature set **F***.

F* CHOSEN BY OUR CODE ON THE COMBINED CORPUS (REAL)

`url_length` `domain_length` `path_length`
`token_count` `num_subdirectories` `special_char_count`
`number_count` `num_dots` `num_slashes` `num_hyphens`
`non_alphanumeric_chars` `has_https` `scheme_http`
`num_delimiters_path` `digit_ratio`

Why shrink? Fewer features → faster, less overfitting, and the model stays interpretable. **Why re-select later?** The best clues change when attackers change tactics — that's the "adaptive" part.

STEP 3 · THE CLASSIFIER

Linear SVM + SGD: draw a line, then nudge it

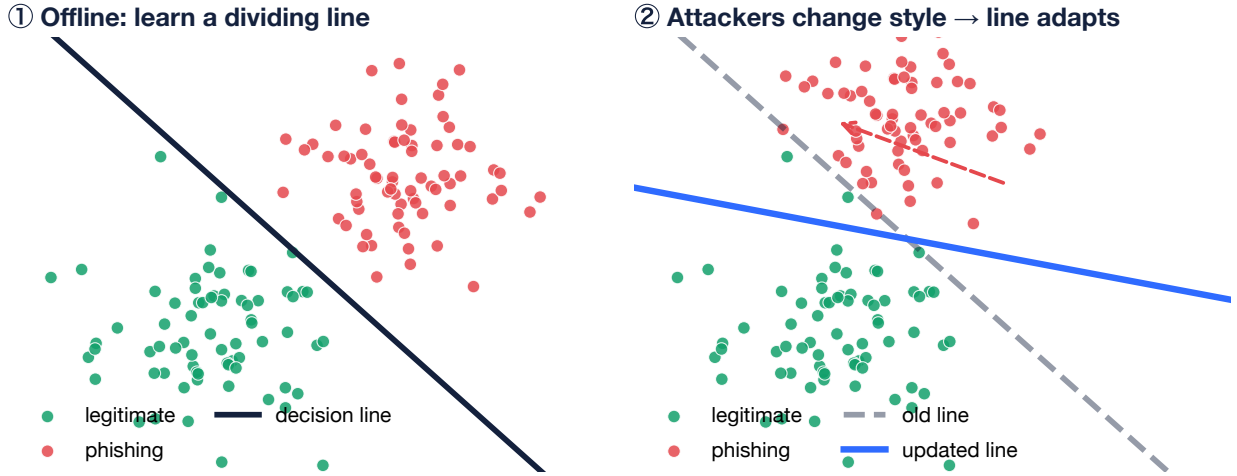


Illustration (toy data). ① Offline: learn a line that separates legit (green) from phishing (red). ② When phishing URLs shift, small mini-batch updates tilt the line — no need to start over.

THREE SGD VARIANTS COMPARED IN THE PAPER

SGD-SVM · hinge loss ★ | **SGD-LR · log loss** | **SGD-Perceptron**

Why "SGD"?

Stochastic Gradient Descent learns from **small chunks** of data. Think of it as *reading one new page* instead of *re-reading the whole library*. That's what makes `partial_fit` on a 500-URL mini-batch possible.

PAPER'S TABLE 4 — WHAT WE USED

`η₀ = 0.01` `adaptive learning rate`
`class_weight = balanced` `early stopping (10 % val.)`
`max_iter = 1000` `LASSO λ = 0.0001` `buffer = 1500`
`mini-batch B = 500` `KS δ = 0.05` `80:20 split`
`seed = 42`

OFFLINE → ONLINE

Offline: fit once on old labelled URLs (batch).
Online: same model keeps updating on each new mini-batch.

STEP 4 · DRIFT DETECTION

KS test: “do these two samples look alike?”

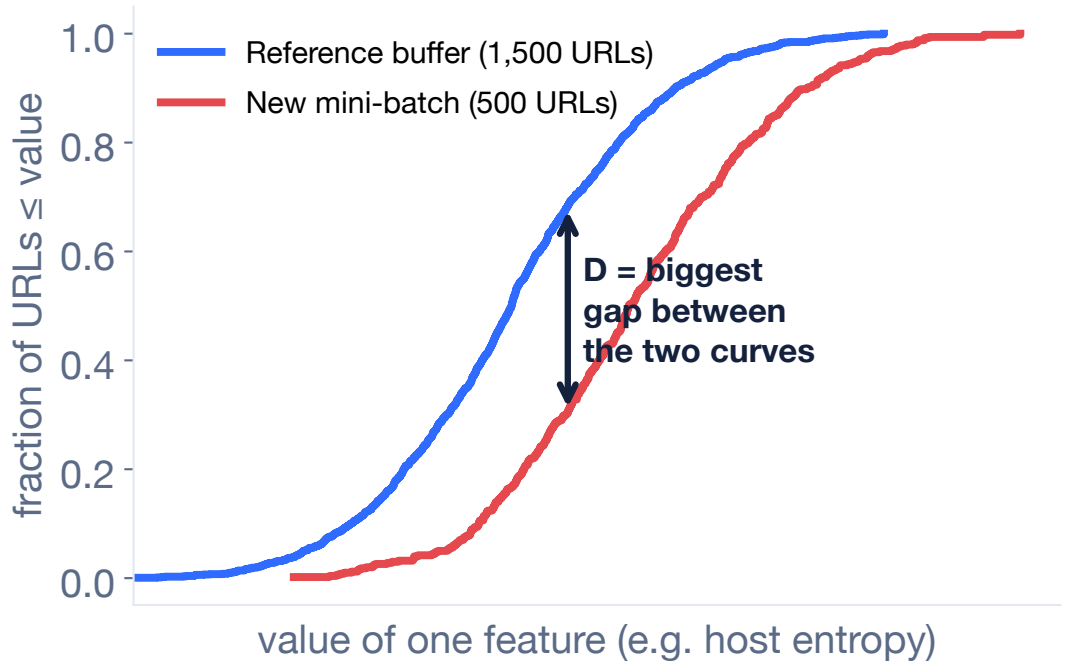


Illustration. For one feature, plot the cumulative distribution of the reference buffer (blue) and the new batch (red). **D** = the largest vertical gap. Big gap ⇒ the distributions differ.

1 **Reference = recent history**
Rolling buffer of the last **1,500** labelled URLs.

2 **Test every feature on every mini-batch**
New batch of **B = 500** URLs vs. the buffer, one KS test per feature.

3 **p-value < δ = 0.05 ⇒ feature has drifted**
“If the two samples came from the same source, a gap this big would be < 5 % likely.”

A CHOICE THE PAPER LEAVES OPEN

Paper doesn’t say how per-feature alarms become one *drift event*. **Our rule:** drift event when **≥ 25 % of the active features F*** fail the KS test.

STEP 5 · PUTTING IT TOGETHER

The online loop: what happens to every mini-batch

500 new URLs
mini-batch arrives

› **1 Predict first**
"test-then-train": score the batch *before* learning

› **2 KS test**
every feature vs. buffer

› **3 Drift event?**
≥ 25 % of F* drifted

› **4 Learn**
SGD *partial_fit* on the batch

If drift is detected — the adaptive part, in 6 steps (`framework.py` , `dynamic.py`)

a. Mark drifted features in F* as **obsolete**

b. Re-run **LASSO + MI** on the 1,500-URL buffer

c. Blend with the stored importance **history H** (50/50) to damp noise

d. New F* = top quarter again: **old features out, new ones in**

e. Model keeps its weights; removed features are masked to 0 — **partial retraining**, 3 replay epochs on the buffer

f. Reset the KS reference to the new buffer

WHY "PREDICT FIRST"?

It's the honest way to measure a streaming model: every URL is scored by a model that has **not** seen it yet (prequential evaluation).

OUR DESIGN CHOICES

25 % trigger · history weight 0.5 · 3 replay epochs · inputs clipped to ±10 · classifier kept in full 58-feature space with F* as a mask.

WHAT WE RAN

Data and the five experiments

Datasets

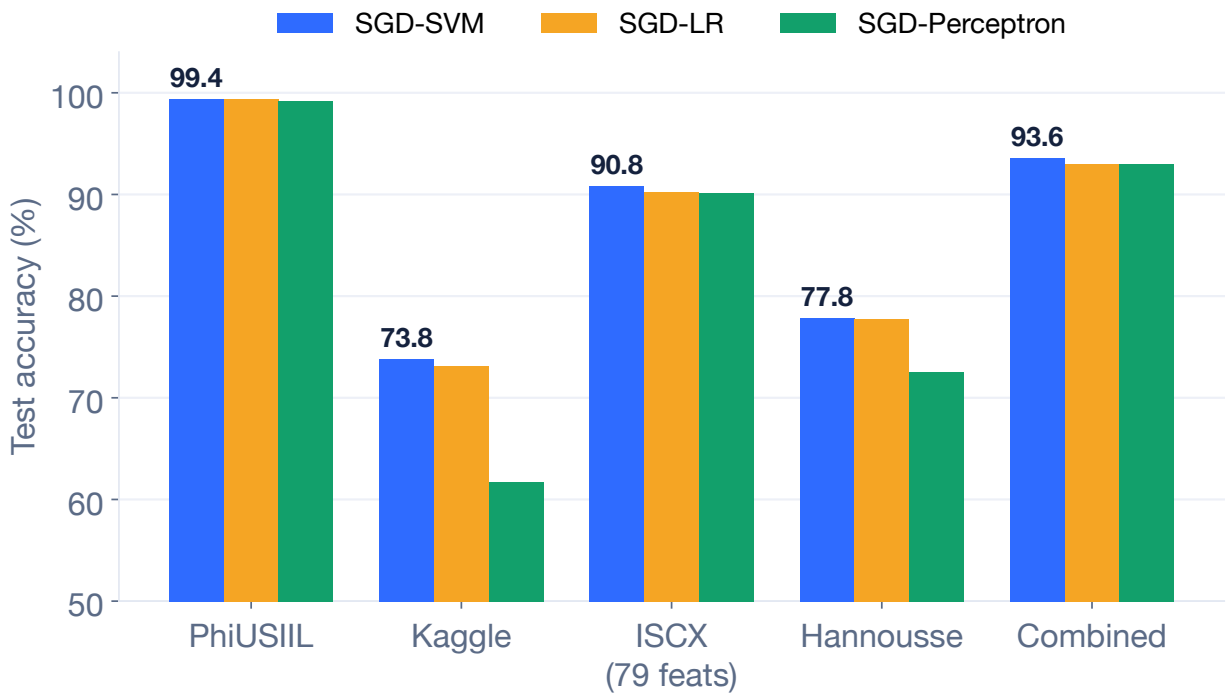
CORPUS	URLS	ROLE
PhiUSIIL	235k	paper's largest corpus (Table 7)
Kaggle	21k	balanced subset (Table 6)
ISCX-URL-2016	15.4k	authors' 79 pre-computed features only (Table 5)
Hannousse 2021	11.4k	raw URLs, used *in place of* ISCX where strings are needed
Combined	263k	PhiUSIIL + Kaggle + Hannousse, de-duplicated
PhishTank stream	**12,888**	live verified phishing, **Jun 2024 – Jan 2025** (paper's window), chronological, + equal held-out benign

1. **Offline** · Tables 5–7, Fig. 6
   SGD-SVM / LR / Perceptron per corpus
2. **Online stream** · Sec. 5 / Figs. 8–9
   S1 PhishTank · S2 strong cross-corpus shift
3. **Ablation** · Table 9
   adaptive vs. static F*, N = 100 / 500 / 1000
4. **Runtime** · Tables 10–11
   latency; incremental vs. batch retraining
5. **Drift analysis** · Figs. 4, 5, 7, Table 8
   KDE, co-occurrence, KS, feature evolution

Seeds & setup: positive class = phishing · 80:20 stratified split, seed 42 · streams: 5 seeds (mean ± std) · mini-batch B = 500 · PhishTank snapshot of 2026-11-06 is stored with the code.

RESULTS 1 · OFFLINE PHASE

SGD-SVM is the best linear model, as in the paper



Our Fig. 6 analogue. Test accuracy of the three SGD variants on LASSO+MI features. SVM (blue) is best or tied on every corpus.

SGD-SVM ACCURACY: THIS STUDY VS. PAPER

CORPUS	OURS	PAPER	
PhiUSIIL (T7)	**99.38**	98.78	✓
ISCX (T5)	**90.83**	99.02	≈
Kaggle (T6)	**73.81**	99.00	≈
Combined (F6)	**93.58**	99.85	≈

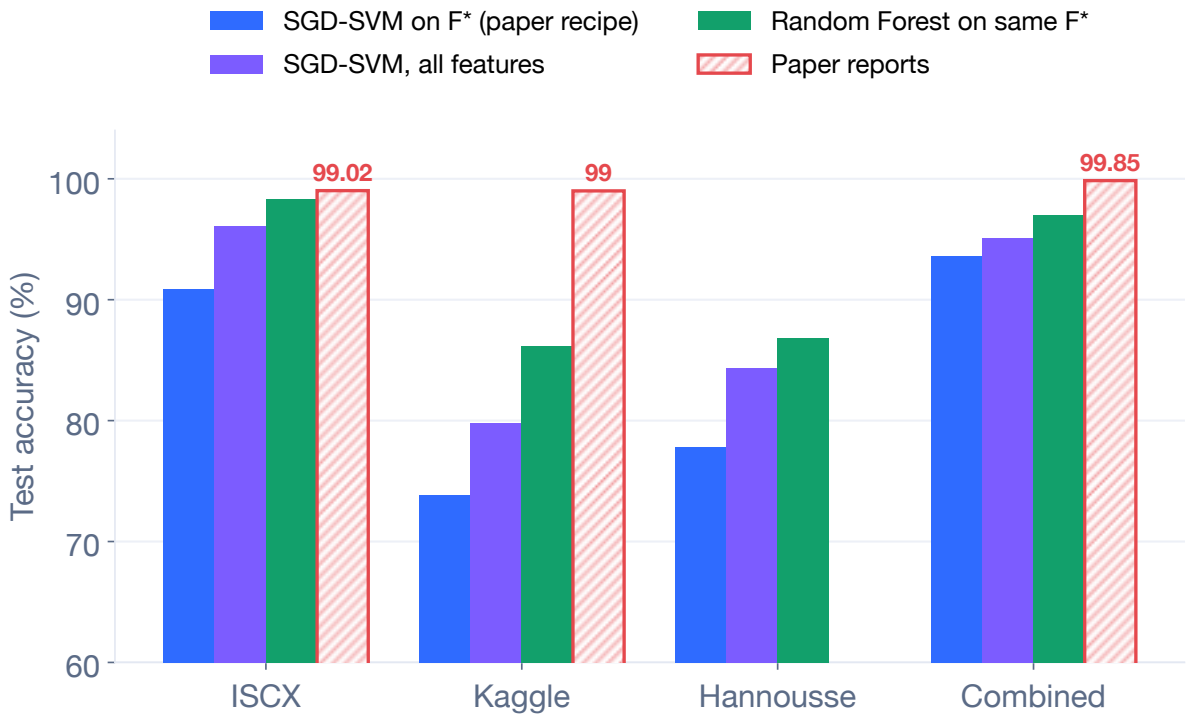
99.38%

PhiUSIIL accuracy — beats the paper's 98.78 %; FPR only 0.004 %

≈ means: same ordering of methods, lower absolute level. Next slide: why.

RESULTS 1 · WHY?

Why our Kaggle / ISCX / combined numbers are lower

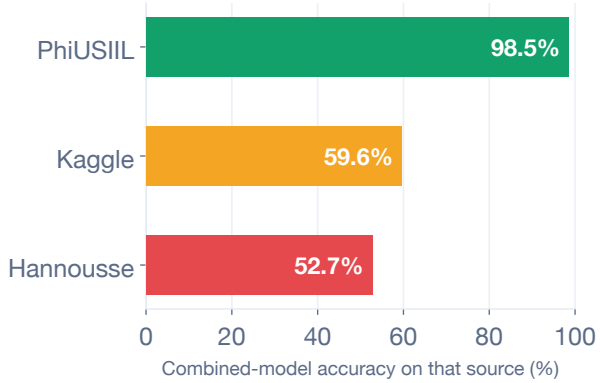


Same learner, different feature choice or model. Using all features lifts SGD-SVM (ISCX 90.8 → **96.1 %**, Kaggle 73.8 → **79.8 %**); a random forest on the *same* F* reaches **98.3 %** on ISCX.

1 Level depends on the feature set
Only the top-quarter F* is used; the paper doesn't publish its exact feature definitions.

2 Data versions & cleaning differ
Paper gives no exact corpus versions or URL pre-processing; ISCX was available to us only as features.

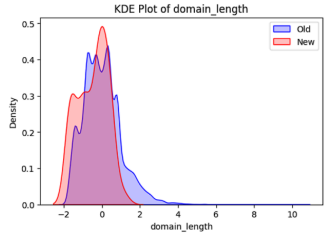
3 Corpora look very different
PhiUSIIL's benign URLs are mostly bare domains → easy. Combined model, per source:



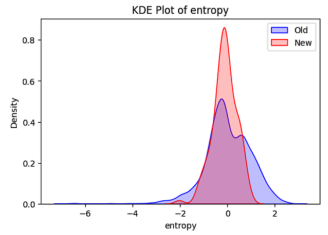
RESULTS 2 · DRIFT IS REAL

Old vs. new phishing URLs: paper's plots and ours

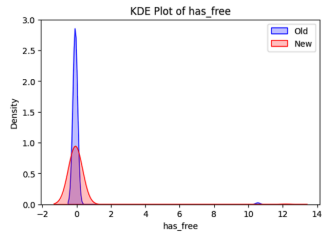
Paper · Fig. 4 (KDE: old blue, new red)



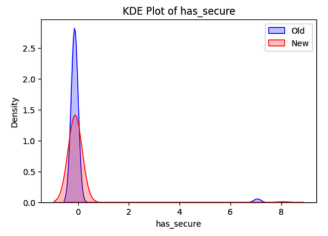
(a) Longer domains replacing traditional short phishing URLs



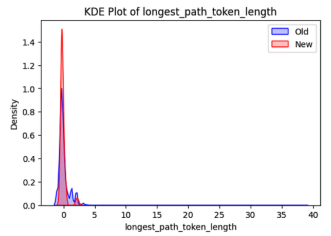
(b) Higher entropy values indicating more random character combinations



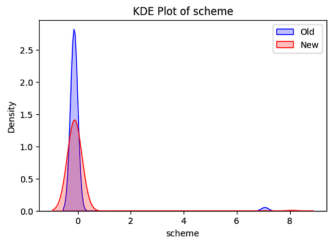
(c) Decreased usage of 'free' as bait in newer URLs



(d) Reduced 'secure' keyword impersonation tactics

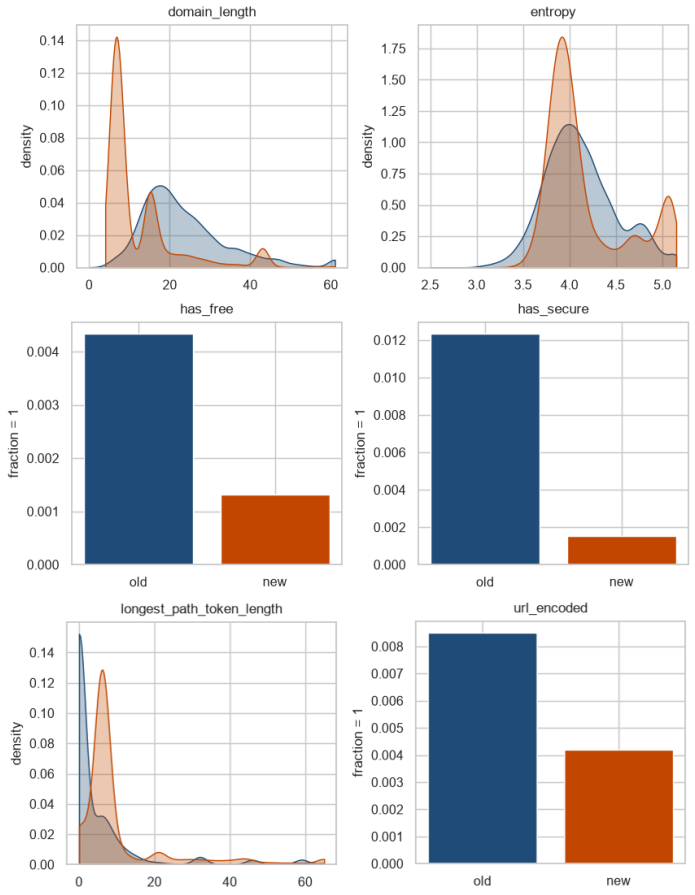


(e) Shorter Path Tokens in Recent Phishing URLs Mimicking Legitimate Site Structures



(f) Rising Adoption of HTTPS in Phishing URLs to Exploit User Trust Indicators

Ours · same features (old gray-blue, new orange)



WHAT BOTH SHOW

- New URLs concentrate in **narrower, different** ranges — e.g. domain length & entropy.
- Bait words `has_free`, `has_secure` are **rarer** in new URLs.

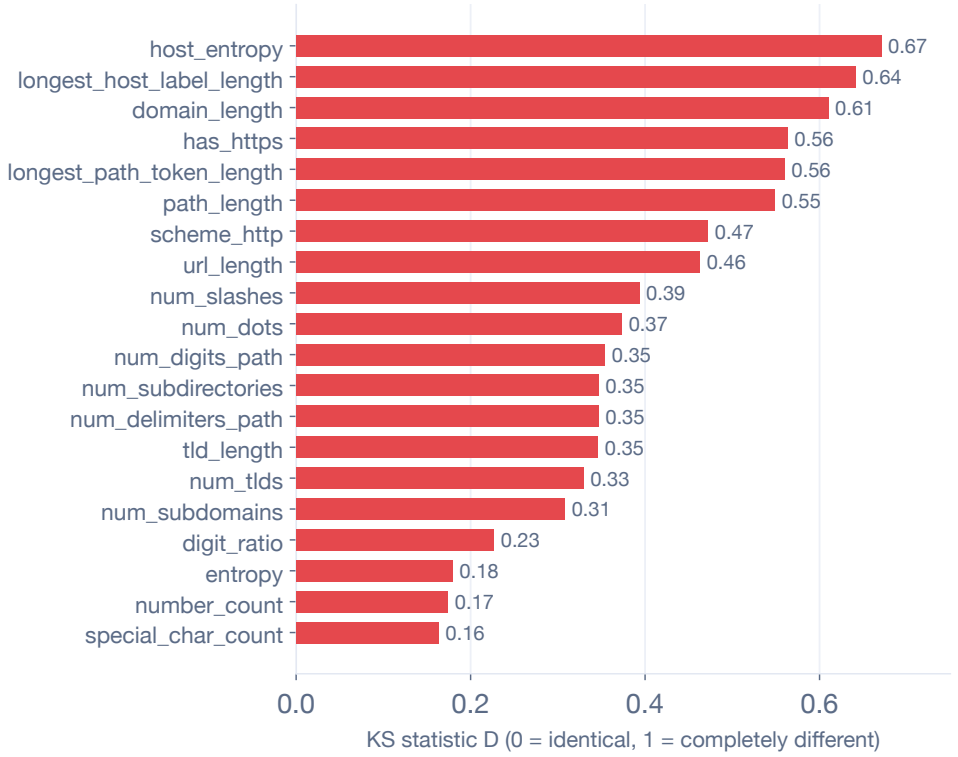
34 / 58

features differ significantly (KS, δ = 0.05) — the paper's core premise holds on real June 2024–Jan 2025 data

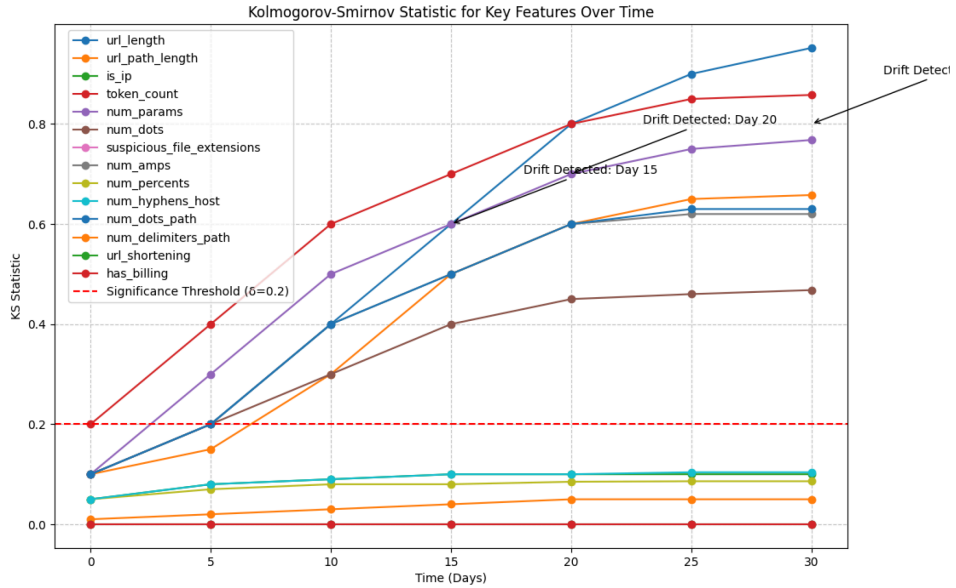
RESULTS 2 · DRIVERS

KS statistic per feature (paper Fig. 7)

Ours · 1,500 old vs 500 new phishing URLs



Paper · Fig. 7 (KS statistic over time)



READING IT

- Longer bar = bigger shift; **red = drifted**, gray = stable.
- Biggest shifts: host entropy, host label & domain length, https/http, path & URL length — the **length / structure** family the paper also reports.

RESULTS 2 · ADAPTATION

Selected features change after drift (paper Table 8)

INITIAL · 15 FEATURES

`url_length` `domain_length` `path_length` `token_count` `num_subdirectories` `special_char_count` `number_count` `num_slashes` `non_alphanumeric_chars` `num_delimiters_path` `digit_ratio` `num_hyphens` `num_dots` `has_https` `scheme_http`

›

DRIFT STAGE 1

+ entropy **+ host_entropy** **+ longest_host_label_length**

~~has_https~~ ~~num_dots~~ ~~scheme_http~~

Host-name **randomness** becomes a key clue.

›

STAGE 2 · 500 URLS

+ num_ampersand

~~num_hyphens~~

Query-string symbols (`&`) start to matter.

›

STAGE 3 · 1,000 URLS

+ num_digits_query

~~num_ampersand~~

Digits *inside the query* replace the symbol count.

What this shows

Each drift event swaps a few features: **+ new clue** in, **old clue** out. In our run the set moves toward **host-name randomness** and **query-string** features — consistent with the paper's story that newer phishing relies on query obfuscation.

avg. drift events, 25,776-URL PhishTank stream

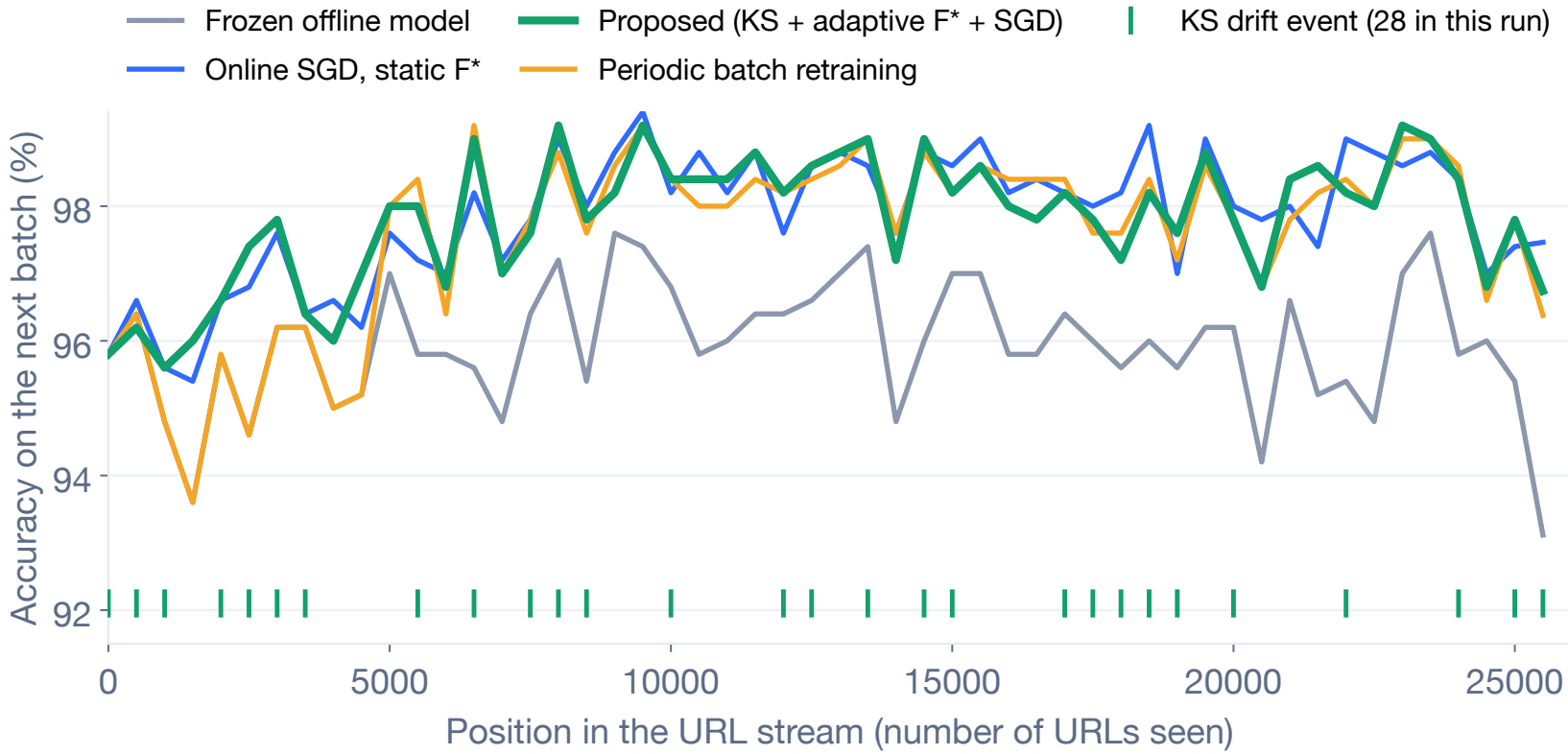
28.6

drift events, cross-corpus stream

7

RESULTS 3 · STREAM S1

Online learning beats a frozen model by +5 points



Frozen offline model
92.63%

Online SGD, static F*
97.77%

Proposed (adaptive)
97.80%

Periodic batch retraining
97.31%

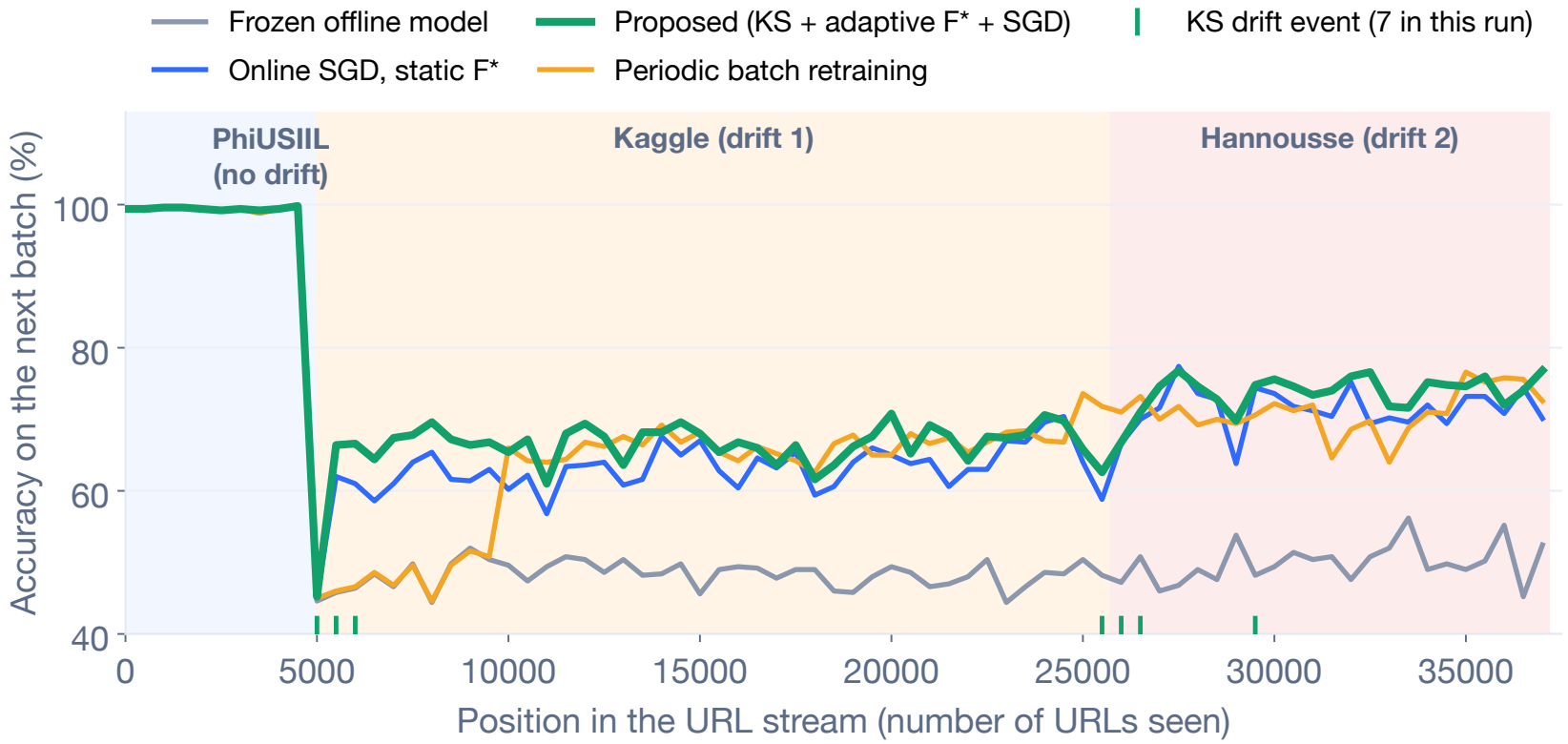
Prequential accuracy per 500-URL batch, 25,776 URLs, Jun 2024 – Jan 2025 (seed 42). Green ticks = KS drift events.

✓ Online learning **+5.1 pts** over frozen, with far lower variance (± 0.05 vs ± 6.4).

≈ Adaptive ≈ static here: this shift is **gentle**. Adaptation matters more under big shifts →

RESULTS 3 · STREAM S2

Under big shifts, adaptive selection really helps



Frozen offline model
55.5%

Online SGD, static F*
70.5%

Proposed (adaptive)
73.2%

Periodic batch retraining
70.0%

S2: train on PhiUSIIL only → stream held-out PhiUSIIL (no drift) → Kaggle (drift 1) → Hannousse (drift 2). Shading = where the URLs come from; green ticks = KS drift events.

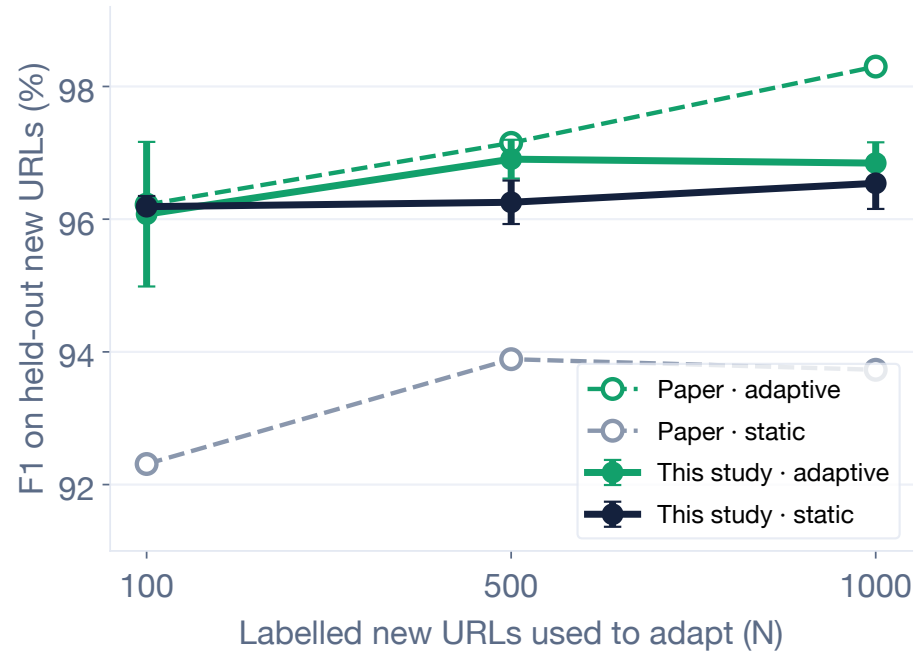
✓ Frozen collapses (≈ 48 % on Kaggle); online recovers; **adaptive is best**: +2.7 pts over static, +15 over frozen.

✓ Periodic batch (orange) must **wait 5,000 URLs** before it reacts — see its long dip after the first shift.

RESULTS 4 · ABLATION (TABLE 9)

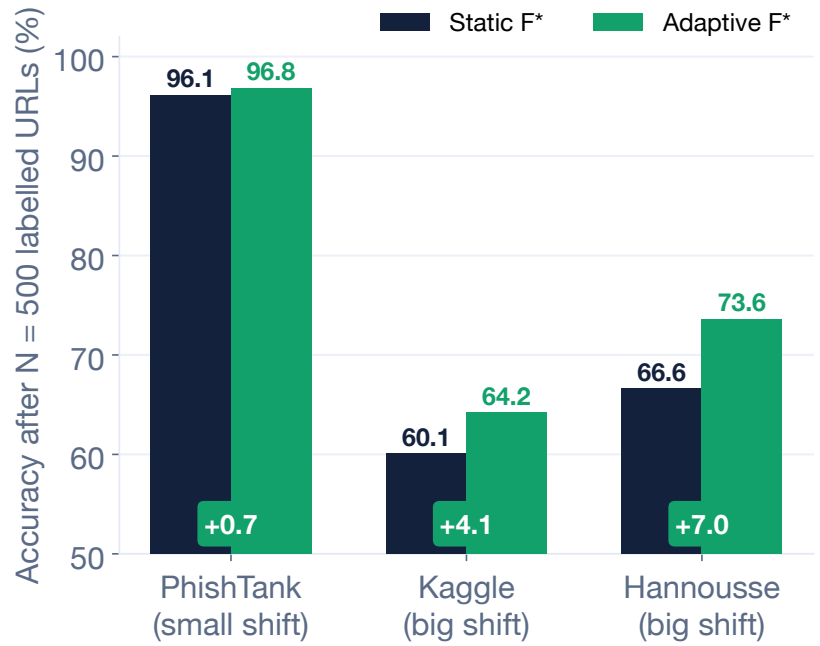
Does adaptive feature selection add value?

F1 vs. number of labelled new URLs (PhishTank target)



Solid = ours (10 seeds, ± std). Dashed = paper. Our **adaptive** F1 (96.1 / 96.9 / 96.9) is within **0.1–1.5 pts** of the paper's (96.2 / 97.2 / 98.3).

Accuracy at N = 500, three targets



Bigger shift ⇒ bigger gain: +0.7 pts on PhishTank, +4.1 on Kaggle, +7.0 on Hannousse. FPR at N ≥ 500 also drops (6.8 % → 5.4 %).

≈ **Caveat:** our static baseline is already stronger than the paper's (96.2 vs 92.3 F1 at N = 100), so the static→adaptive gap on PhishTank is smaller than the paper's.

RESULTS 5 · RUNTIME (TABLES 10–11)

Real-time per URL, and cheap to update

0.16 ms

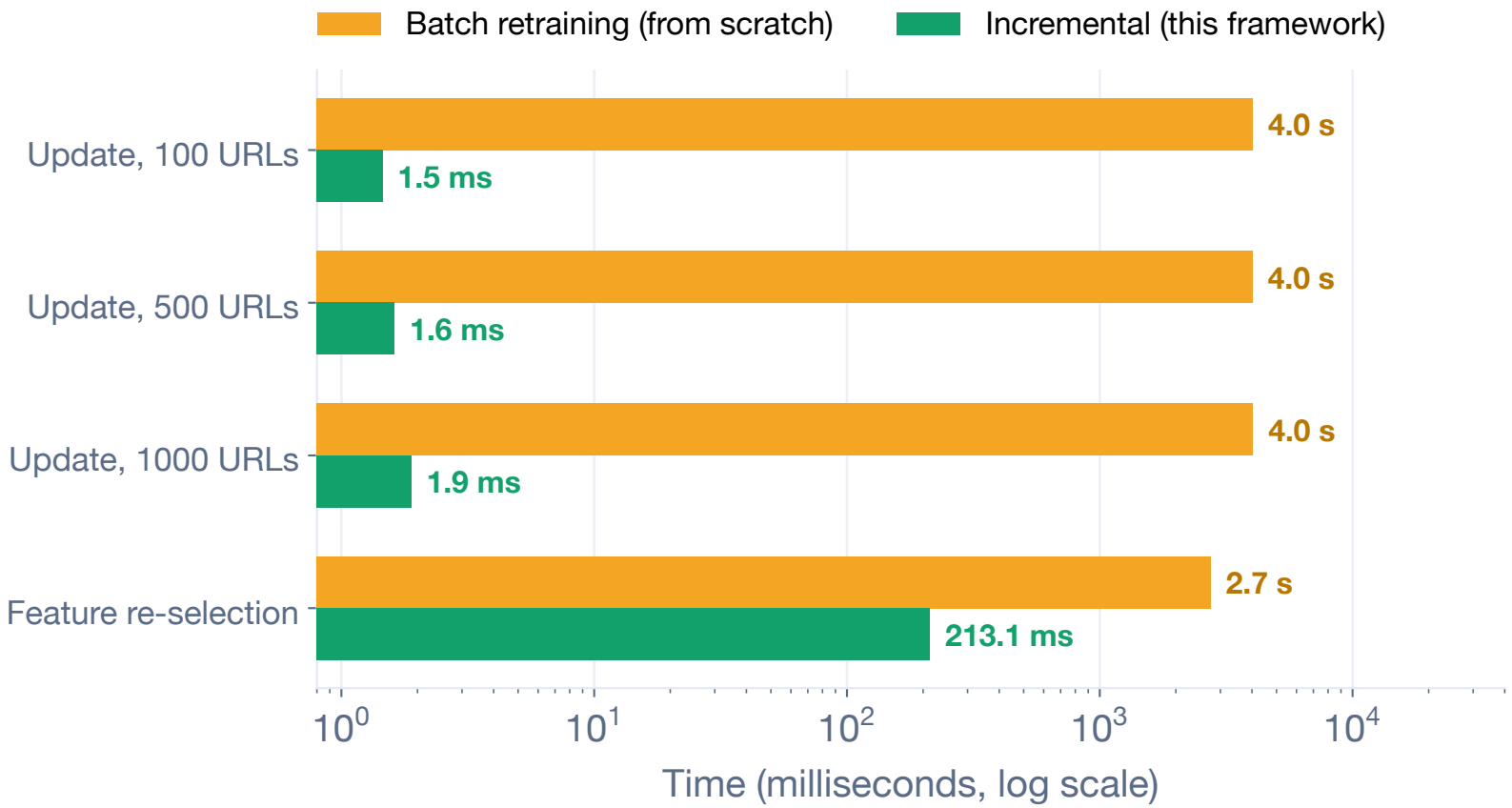
per URL: parse → 58 features → SGD-SVM score (1 CPU thread, 6,000 URLs). Paper: 12.4–15.2 ms on different hardware.

2,000×+

cheaper to update the model with 100–1,000 URLs than to retrain on 210k URLs (paper: 15.9–33.7×)

12.9×

cheaper to re-select features incrementally (213 ms vs 2.7 s; paper: 30.9×)



Mean of 10 runs, log scale. Batch retraining = LASSO + MI + full SGD fit on 210,598 + N URLs.

✓ Same conclusion as the paper: **incremental ≪ retraining**. Absolute numbers differ (hardware, implementation, and we compare against our own batch baseline).

SCORECARD

Paper's claims vs. what we reproduced

PAPER CLAIM / EXPERIMENT	OUR RESULT	STATUS
SGD-SVM is the best of the three SGD variants (T5–7, F6)	best / tied on all 5 corpora	✓ reproduced
PhiUSIIL accuracy (T7: 98.78 %)	**99.38 %**, precision 99.99 %, FPR 0.004 %	✓ reproduced
URL feature distributions drift between old and new (F4, F7)	34 / 58 features drift (KS, $\delta = 0.05$); same feature families	✓ reproduced
Selected feature set evolves after drift (T8)	additions / removals logged at every drift event	✓ reproduced
Online learning beats a frozen model	+5.1 pts (PhishTank), +15.0 pts (cross-corpus)	✓ reproduced
Adaptive features raise F1 (T9: 96.22 / 97.15 / 98.30)	96.08 / 96.90 / 96.85 — gain clearest under big shifts	✓ close
Incremental update ≪ batch retraining (T11: 15.9–33.7×)	2,140–2,770× (update), 12.9× (features)	✓ reproduced
Real-time per-URL latency (T10: 12.4–15.2 ms)	0.16 ms	✓ real-time
≈ 99 % accuracy on ISCX / Kaggle / combined (T5, T6, F6: 99.85 %, FPR 0.067 %)	90.8 % / 73.8 % / 93.6 % (FPR 6.2 %)	≈ lower level

Bottom line: the method and every *qualitative* finding reproduce. The paper's ≈ 99 % absolute levels depend on unpublished data versions and feature definitions we could not recover.

DEMO

PhishCheck: the detector running in a browser

Phishing-style URL → flagged

Search URL

http://secure-paypal-login.tk/confirm/account?user=1&id=88

Search

Press Search or Enter. Scoring uses URL text only.

www.wikipedia.org | github.com | paypa1-secure-login.tk/confi... | 192.168.1.10/login/verify-ac...

Phishing phishing probability 100.0%

Legitimate URL → passes

Search URL

https://github.com

Search

Press Search or Enter. Scoring uses URL text only.

www.wikipedia.org | github.com | paypa1-secure-login.tk/confi... | 192.168.1.10/login/verify-ac...

Legitimate phishing probability 48.0%

HOW IT WORKS

1. Python trains an SGD-SVM and exports its weights to `model.json`.
2. A TypeScript port of the feature extractor runs **in your browser**.
3. Score = weights · features → verdict + probability.

`Source_Code/web/` (Next.js). The URL text never leaves your machine.

Honest note. Bare-domain legitimate URLs sit near the decision boundary: github.com scores 48 % (passes), but wikipedia.org scored 51.8 % — a false alarm. A URL-only model has limited context; the paper itself lists this limitation.

WRAP-UP

Takeaways, limitations and what's next

What we learned

- Phishing URLs **really do drift** — 34 of 58 features shifted.
- A frozen model **decays** (99 % → 48 %); online learning recovers it.
- Adaptive feature re-selection **helps most under big shifts**.
- Incremental updates are **milliseconds**, not seconds.

Limitations

- URL-only features miss attacks with **clean-looking URLs**.
- Absolute accuracy depends heavily on **dataset & feature versions**.
- Drift rule (25 %), history weight, replay epochs are **our choices**.
- PhishTank has only phishing; benign side of the stream is held-out old URLs.

Future work

- Selective **lightweight content checks** for ambiguous URLs (paper's own suggestion).
- Real live-labelled feed & delayed-label handling.
- Stronger / non-linear online learners.

We implemented the full pipeline from the paper's description, reproduced its qualitative findings, and quantified honestly where the absolute numbers differ and why.

Thank you — questions?

Pratham Garg · Sumit Sharma · Vivaan Garg — CS 552 Group Project

ASSIGNED PAPER

Sruthi Krishnan and Manohar Naik S. “An Efficient Framework for Phishing URL Detection Using Adaptive Drift-Aware Feature Monitoring and Online Learning.” *Security and Privacy*, vol. 8, no. 4, article e70057, 2025. John Wiley & Sons Ltd. DOI: 10.1002/spy2.70057

https://onlinelibrary.wiley.com/doi/full/10.1002/spy2.70057

REPRODUCE EVERYTHING

```
cd Source_Code
pip install -r requirements.txt
make test   # 16 unit tests
make all    # data → exp1…5 → results
```

Code: `Source_Code/` · Results: `Experimental_Results/` · Demo: `Source_Code/web/`

99.38%
PhiUSIIL accuracy

+15 pts
online vs. frozen under big shift

34 / 58
features drift old → new

2,000×+
cheaper than batch retraining

APPENDIX A

Algorithm 1 (paper) ↔ our code

PAPER'S ALGORITHM 1 — ADAPTIVE FEATURE SELECTION

Step 1 Initial selection

```
  w = |Lasso.coef|   I = MI(X_B, y_B)   S = w + I
```

Step 2 Drift monitoring

```
  while new batch arrives:
    for each f in F*: KS test → if drift: mark f obsolete, remove
    τ = Percentile(S, 75)   F* = {f : S(f) ≥ τ}
    store S in history H
return F*
```

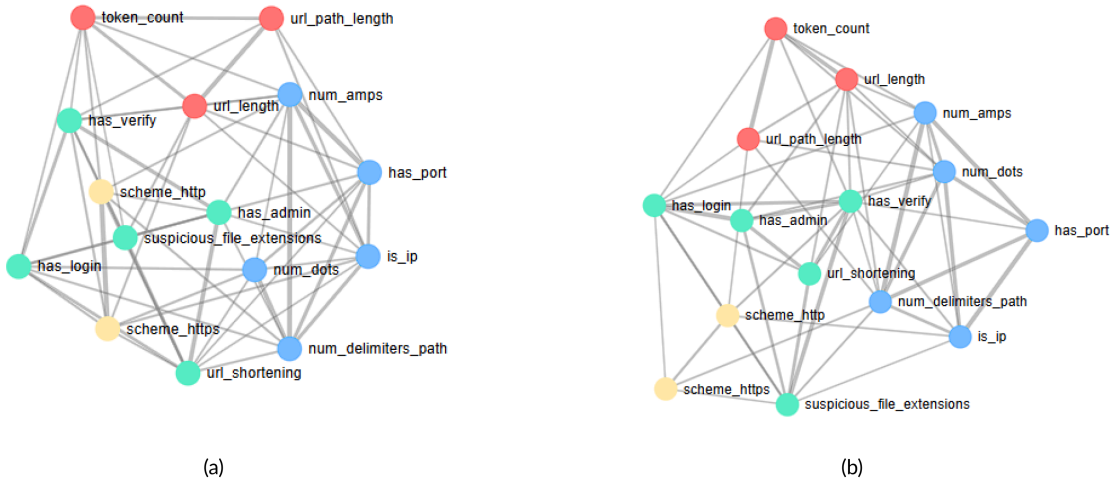
OUR IMPLEMENTATION

`w, mi = _score(X, y)`	Lasso(α=1e-4, 1000 it.) + *mutual_info_classif* on standardised X
`S = w + mi`	`fit()` — initial F*
`ks_2samp`	`KSDriftMonitor.test()` per feature
`np.percentile(S, 75)`	`_apply_threshold()`
`history_`	`update()`: S = 0.5·S_new + 0.5·S_prev
obsolete / added	returned by `update()` → logged as Table 8

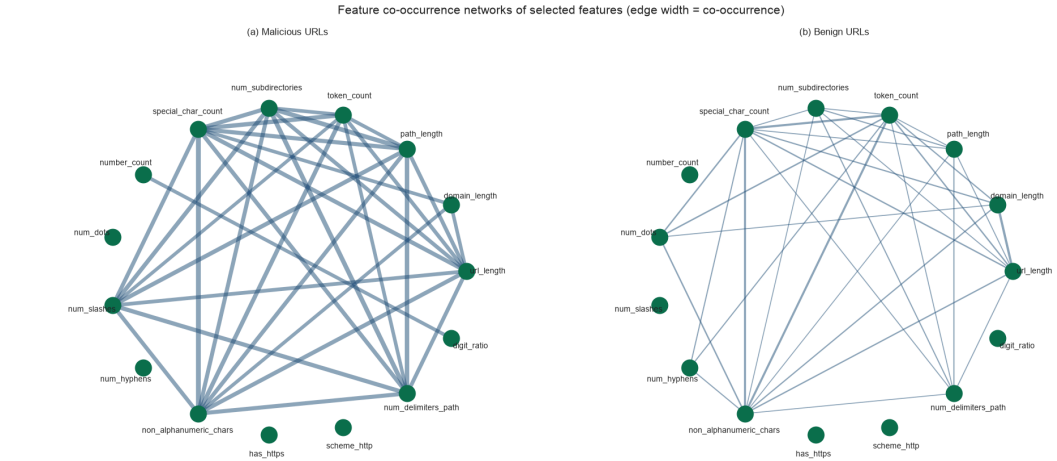
APPENDIX B

Feature co-occurrence networks (paper Fig. 5)

Paper · (a) malicious, (b) benign



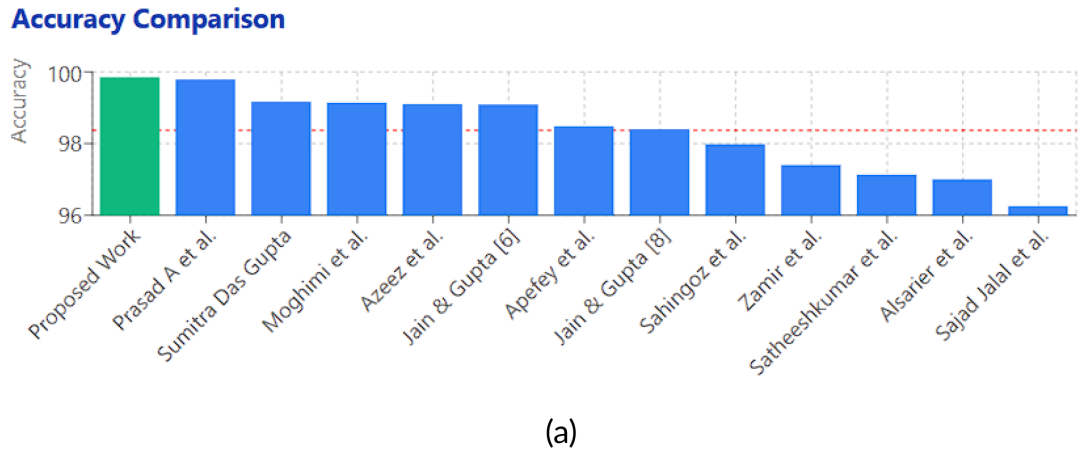
Ours · (a) malicious, (b) benign



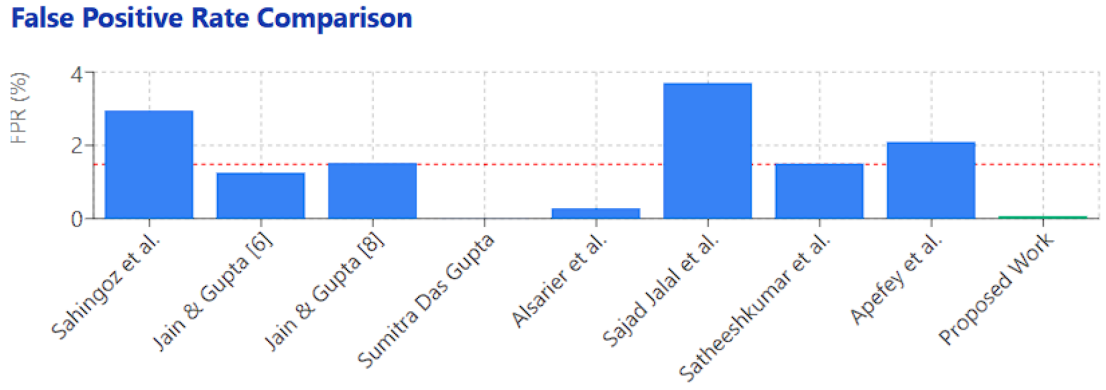
Reading it: each node is a selected feature; edge thickness = how often two features are "on" together. **Paper:** malicious URLs form a dense cluster among security-sensitive features (login, admin, verify, file extension); benign links are more evenly spread. **Ours:** malicious URLs show far thicker and more numerous edges than benign ones — phishing URLs tend to trip several signals at once. (Our F* has different members, so nodes differ.)

APPENDIX C

What the paper compares against (paper Fig. 10)



(a)



(b)

PAPER'S ARGUMENT

Highest accuracy (99.85 %) **and** lowest FPR (0.067 %) among 12 published methods — while using only URL strings and updating online.

OUR VIEW

We did not re-run the other 12 methods. On our data, a batch random forest on the same features beats SGD-SVM offline (e.g. 98.3 % vs 90.8 % on ISCX) — the paper's advantage is **adaptivity and cost**, not raw offline accuracy.